SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Pseudocode Writing Task

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO5 – Naturalization (BL5, BL6)	**Assessment:**	Assessment Tool 2 – Pseudocode Writing Task
Weightage:	30% of CIA	**Total Marks:**	100
CO5 Statement:	*Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.*		
Student Name:	**K. HEMASREE**	**Reg. No.:**	**192411156**
Section / Batch:	**SLOT C**	**Date:**	**27/08/26**

Instructions to Students

- Answer the following question with proper pseudocode and logical explanation.
- Clearly specify the algorithmic approach and complexity class wherever applicable.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) for complexity analysis.
- Include state-space representation, recursion steps, and pruning conditions in backtracking problems. • Neat presentation and step-by-step justification are mandatory.

Question Type	Focus Area	Questions
Pseudocode Writing Task	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP-Complete.	Q1

SECTION A – Pseudocode Writing Task

Code of Conduct

I K.HEMASREE(192411156)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning

Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Marks Evaluation:

Criteria	Wt.	Lev Exer
Problem Understanding	20%	
Algorithm Design	30%	
Use of Control Structures	20%	
Pseudocode Structure	10%	
Completeness	20%	
Total Marks (100):		

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

AIM

To develop a **backtracking algorithm for the Graph Partition Problem** that divides a graph with an even number of vertices into **two equal-sized disjoint subsets**, while maintaining balance and eliminating invalid partitions using pruning.

ALGORITHM

1. **Start** the algorithm.
2. Take the input graph **G = (V, E)**, where the number of vertices n is even.
3. Initialize two empty subsets:
    - $A = \varnothing$
    - $B = \varnothing$
4. Set the maximum allowed size of each subset to $n/2$.
5. Start the **backtracking process** from the first vertex.
6. For the current vertex, try assigning it to subset **A**.
7. If the size of A does not exceed $n/2$, recursively process the next vertex.
8. Backtrack and try assigning the current vertex to subset **B**.
9. If the size of B does not exceed $n/2$, recursively process the next vertex.
10. **Pruning:** If either $|A| > n/2$ or $|B| > n/2$, immediately stop that branch because it cannot produce two equal subsets.
11. When all vertices are assigned:
    - Check whether $|A| = |B| = n/2$.
    - If valid, calculate the cut size or required partition measure.
12. Store the best valid partition found.
13. Continue backtracking until all possible valid assignments are considered.
14. Return the two subsets **A and B** and the best partition/cut value.
15. **Stop**.

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

KEERTHIPATI HEMA SREE
192411156RC

CO5 AT2-Pseudocode Writing Task (Algorithm Design)

← Back

Language: Python | C | C++ | Java

QUESTIONS

Q1
Graph Partition Problem
100 marks

1/1 answered

Q1 Graph Partition Problem — 100 marks

Write a pseudocode using Backtracking to partition a graph into two equal sets. Clearly define inputs (graph) and output (two subsets). Design an algorithm that assigns vertices to partitions. Use control structures to maintain balance. Apply pruning to eliminate invalid partitions. Present the pseudocode clearly. Ensure completeness.

Your Code

```
ALGORITHM BalancedGraphPartition(G)
    // Input: A graph G = (V, E) with an even number of vertices n = |V
    // Output: Two disjoint subsets A and B such that A ∪ B = V and |A

    n ← length(V)
    A ← empty set
    B ← empty set
    best_cut_size ← infinity
    best_A ← empty set
    best_B ← empty set

    // Start backtracking from the first vertex (index 0)
    BacktrackPartition(0, G, A, B, n, best_cut_size, best_A, best_B)

    return best_A, best_B, best_cut_size
END ALGORITHM


PROCEDURE BacktrackPartition(index, G, A, B, n, ref best_cut_size, ref
    // Control Structure / Pruning: Eliminate invalid partitions early
    // If either subset grows larger than n/2, it cannot form a balance
    if length(A) > n/2 OR length(B) > n/2 then
        return
    end if
```

Input

stdin input

Output

```
  File
"C:\Windows\TEMP\sandbox_1924111
56RC_6a8fb0b07af568.59108356\mai
n.py", line 3
    // Output: Two disjoint
subsets A and B such that A
```